

任务11：聊天会话记忆与历史上下文管理

在真实的企业级项目中，单轮问答只能完成最基础的聊天能力，但用户往往希望系统能够记住前文上下文，支持连续追问。例如，用户先问“什么是违约责任”，下一句再问“那买卖合同里通常怎么约定”，如果系统不能记住上一轮语境，对话体验会明显下降。Spring AI 官方提供了 Chat Memory 机制，用于跨多轮对话保留上下文。

本任务将实现 聊天会话记忆与历史上下文管理，下一次任务实现 知识文档上传与解析。

1. 设计会话数据结构

1.1 扩展聊天请求对象

在任务10的基础上，修改 `ChatRequestDTO`，让聊天请求不再只包含 `message`，还包含：

- `sessionId`
- `message`

文件位置：

```
/model/dto/ChatRequestDTO.java
```

代码示例：

```
package com.example.demo.model.dto;

import lombok.Data;

@Data
public class ChatRequestDTO {

    /**
```

```
    * 会话编号，用于标识同一轮连续对话
    */
    private String sessionId;

    /**
     * 当前用户输入
     */
    private String message;
}
```

说明：

`sessionId` 可以由 Postman 手动填写。
当前任务先不要求自动生成会话编号。

1.2 新建聊天记录对象

建议新建 `ChatRecord`，用于保存每一轮会话记录。

文件位置：

```
/model/entity/ChatRecord.java
```

代码示例：

```
package com.example.demo.model.entity;

import lombok.Data;

import java.time.LocalDateTime;

@Data
public class ChatRecord {

    /**
     * 会话编号
     */
    private String sessionId;
```

```
/**
 * 用户问题
 */
private String userMessage;

/**
 * 大模型回答
 */
private String assistantMessage;

/**
 * 记录时间
 */
private LocalDateTime createTime;
}
```

说明：

当前任务中 `ChatRecord` 可以先作为缓存对象使用，不强制要求立刻持久化到数据库表。

如果想扩展成数据库存储，可自行增加 `id` 字段和表映射注解。

2. 设计聊天记录存储方式

2.1 先使用 Redis 存储聊天记录

本任务建议优先使用 Redis 来做短期会话缓存。

- 聊天上下文属于“短期记忆”
- Redis 读写快，便于演示多轮对话

建议使用如下 key 规则：

```
chat:session:{sessionId}
```

例如：

```
chat:session:test001
```

2.2 在 Service 中加入会话读取与写入逻辑

每次提问时，处理流程为：

1. 先根据 `sessionId` 读取历史聊天记录
2. 只取最近若干轮（例如最近 3 轮）
3. 将历史上下文与当前问题拼接后发送给模型
4. 得到模型回答后，再把这一轮记录写回 Redis

3. 改造聊天 Service

3.1 先修改 Service 接口

文件位置：

```
/service/ChatService.java
```

将接口方法改为：

```
package com.example.demo.service;

import com.example.demo.model.dto.ChatRequestDTO;
import com.example.demo.model.vo.ChatResponseVO;

public interface ChatService {
    ChatResponseVO chat(ChatRequestDTO requestDTO);
}
```

3.2 改造 `ChatServiceImpl`

文件位置：

```
/service/impl/ChatServiceImpl.java
```

建议在 `ChatServiceImpl` 中注入：

```
private final ChatClient chatClient;  
private final StringRedisTemplate stringRedisTemplate;
```

代码示例：

```
@Service  
public class ChatServiceImpl implements ChatService {  
  
    private final ChatClient chatClient;  
    private final StringRedisTemplate stringRedisTemplate;  
  
    public ChatServiceImpl(ChatClient.Builder chatClientBuilder,  
                           StringRedisTemplate stringRedisTemplate) {  
        this.chatClient = chatClientBuilder  
            .defaultSystem("你是一名专业、友好、简洁的中文智能助手，请结合历史  
            .defaultOptions(  
                DashScopeChatOptions.builder()  
                    .withTopP(0.7)  
                    .build()  
            )  
            .build();  
        this.stringRedisTemplate = stringRedisTemplate;  
    }  
  
    @Override  
    public ChatResponseVO chat(ChatRequestDTO requestDTO) {  
        String sessionId = requestDTO.getSessionId();  
        String message = requestDTO.getMessage();  
  
        String redisKey = "chat:session:" + sessionId;  
  
        // 1. 读取历史消息  
        List<String> records = stringRedisTemplate.opsForList().range(redisKey,  
  
        String historyText = "";
```

```
if (records != null && !records.isEmpty()) {  
    historyText = String.join("\n", records);  
}
```

```
// 2. 拼接上下文  
String finalPrompt = ""  
    以下是历史对话：  
    %s
```

```
// 2. 拼接上下文  
String finalPrompt = ""  
    以下是历史对话：  
    %s  
  
    当前用户问题：  
    %s  
    """.formatted(historyText, message);
```

```
// 3. 调用模型  
String answer = chatClient.prompt(finalPrompt)  
    .call()  
    .content();
```

```
// 4. 保存本轮记录  
String recordText = "用户: " + message + "\n助手: " + answer;  
stringRedisTemplate.opsForList().rightPush(redisKey, recordText);  
  
// 5. 只保留最近 3 轮  
Long size = stringRedisTemplate.opsForList().size(redisKey);  
if (size != null && size > 3) {  
    stringRedisTemplate.opsForList().trim(redisKey, size - 3, size - 1)  
}  
  
return new ChatResponseVO(message, answer);
```

附加题：

- 异常处理

- 历史轮数控制策略
- sessionId 为空时的校验
- Chat Memory 官方实现方式的进一步替换

4. 改造聊天控制器

4.1 修改 ChatController

文件位置：

```
/controller/ChatController.java
```

代码示例：

```
@RestController
@RequestMapping("/api/chat")
public class ChatController {

    private final ChatService chatService;

    public ChatController(ChatService chatService) {
        this.chatService = chatService;
    }

    @PostMapping
    public Result<ChatResponseVO> chat(@RequestBody ChatRequestDTO requestDTO)
        ChatResponseVO responseVO = chatService.chat(requestDTO);
        return Result.success(responseVO);
    }
}
```

5. 联调与测试

5.1 单轮测试

请求地址示例：

```
POST http://localhost:8080/api/chat
```

请求体示例：

```
{
  "sessionId": "test001",
  "message": "这是服务端课程，你是谁？"
}
```

要求：

- 能正常返回答案
 - 不影响原有基础聊天功能
-

5.2 多轮测试

仍然使用相同 `sessionId`：

```
{
  "sessionId": "test001",
  "message": "之前和你聊过什么？根据聊天历史，这是什么课程？"
}
```

要求：

- 模型能结合上一轮“违约责任”的语境继续回答
 - 表现出一定的连续上下文理解能力
-

提交要求

1. 测试与任务成功示例:

截取 单轮聊天成功截图、相同 `sessionId` 连续两轮聊天成功截图。

2. **文档提交:** 将 更新后的核心代码推送至个人仓库，并将截图粘贴至 Word 文档按时提交。